CHAPTER 1 INTRODUCTION TO DESIGN PATTERNS

Software Design Patterns

CPIT-251

What is a Design?

- Formulate the maximum alternative solutions of the problem and select the best one

- Software Design is the process of implementing a software solutions to do one or more set of problems

Why We Need A Design?

- Provides an overall picture of the system
- An understandable design is the best hope to construct a program that works
- Provide facilitate reuse (NOT code reuse)
- Design uses abstraction to create views that are comprehensible (understandable)

Design and Quality

- The design must implement all of the explicit (clear) requirements contained in the analysis model, and it must accommodate all of the implicit (hidden) requirements desired by the customer

- The design must be readable, understandable guide for those who generate code and for those who test and subsequently support the software

- The design should provide a complete picture of the software, addressing the data, functional, and behavioral domains from an implementation perspective

Design Techniques

- Structured/functional Design: data flow diagram (DFD), entity relation diagram (ERD), and Hierarchical Charts
- Object Oriented Design: UML, Use Cases and Class Diagrams
- Examples of Design Tools
  - Rational Rose, Star UML, Together Soft, and Microsoft Visio

Object Oriented Design Techniques

- Basic components
  - Classes, which include objects
  - Functions

- Component relationships
  - One class uses other classes to fulfill its responsibilities
    - Methods from one class call methods in the other classes
  - Inheritance
  - Classes grouped into packages/subsystems

Object Oriented Design Techniques

- Identify basic objects/concepts in application domain, and use these as foundation classes
- Add additional classes for desired functionality
- Abstract out super classes to facilitate understanding and possible re-use
- Refactor as necessary

Definitions

- A *pattern* is a recurring solution to a standard problem, in a context
- Pattern is a structure or template that is designed to provide solution of a specific problem
- “A pattern describes a problem which occurs over and over again in our environment, and then describes the core of the solution to that problem, in such a way that you can use this solution a million times over, without ever doing it the same way twice”

Main Types of Patterns

- Architectural Patterns
- Design Patterns
- Idiom Patterns

- Our main focus is : Design Patterns

Design Pattern

- The design patterns in this book are descriptions of communicating objects and classes that are customized to solve a general design problem in a particular context
- A design pattern identifies the key aspects of a common design structure that can be used to create a reusable object-oriented design
- The design pattern identifies the participating classes and instances, their roles and collaborations, and the distribution of responsibilities
- Each design pattern focuses on a particular object-oriented design problem

Benefits of Design Patterns

- Design patterns enable large-scale reuse of software architectures and also help document systems
- Design patterns explicitly capture expert knowledge and design tradeoffs and make it more widely available
- Design patterns help improve developer communication

Design Pattern Elements

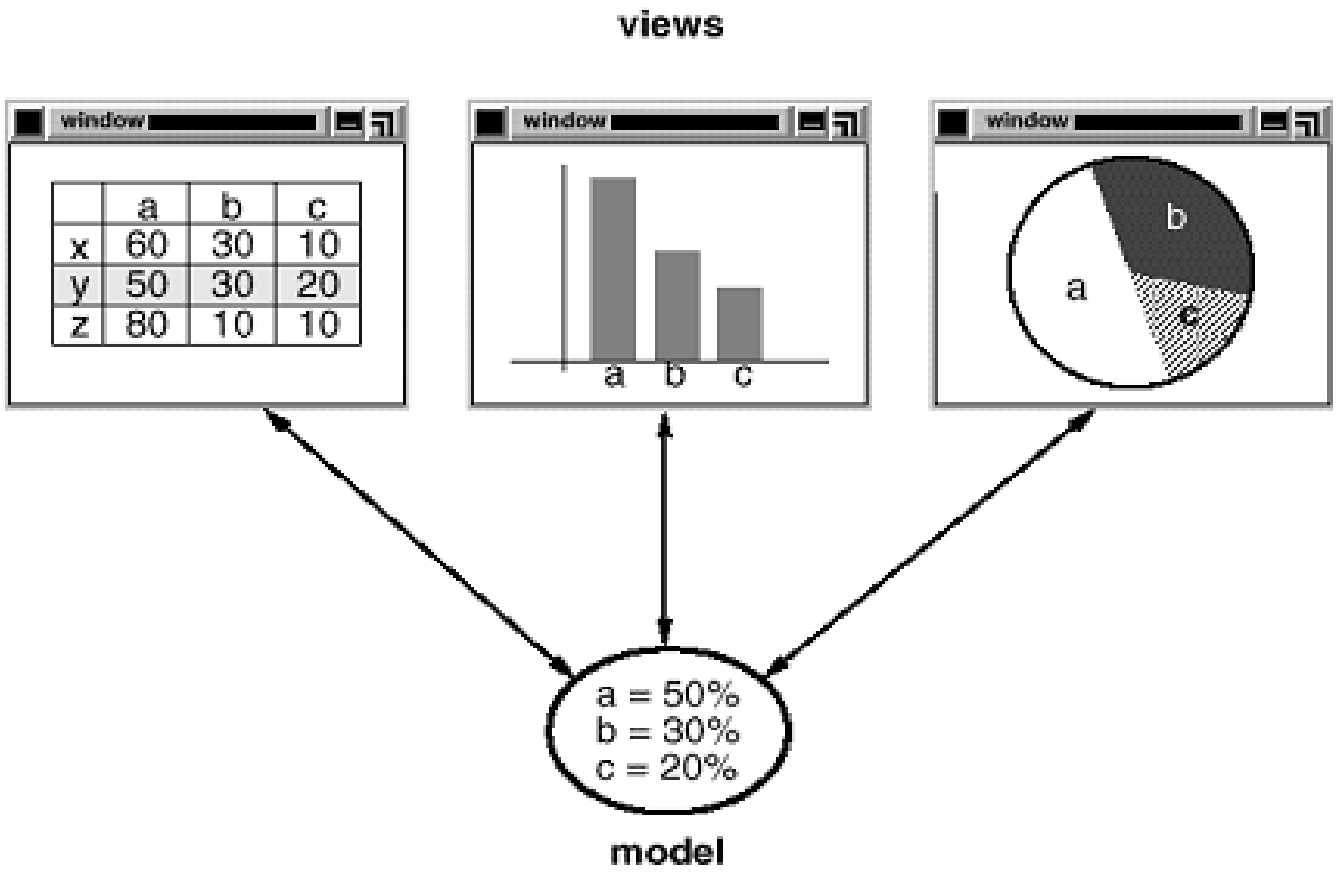
- Design patterns have 4 essential elements:
  - Pattern name
    - Describes a design problem and its solutions
    - Increases vocabulary of designers
  - Problem
    - Describes when to use the pattern
  - Solution
    - Describes the elements that make up the design, their relationships, responsibilities and collaboration
    - UML-like structure, abstract code
  - Consequences
    - The results and tradeoffs of applying them

Model View Controller (MVC)

- MVC is a design pattern used in software engineering to separates the application logic (model) from business logic (view)

- MVC consists of
  - The model, which is the application object
  - The view, which is the screen representation
  - Controller, which defines the way user interface acts to the user input

MVC


views
window
a b c
x 60 30 10
y 50 30 20
z 80 10 10
window
a b c
window
a
b
c
a = 50%
b = 30%
c = 20%
model

How MVC Works?

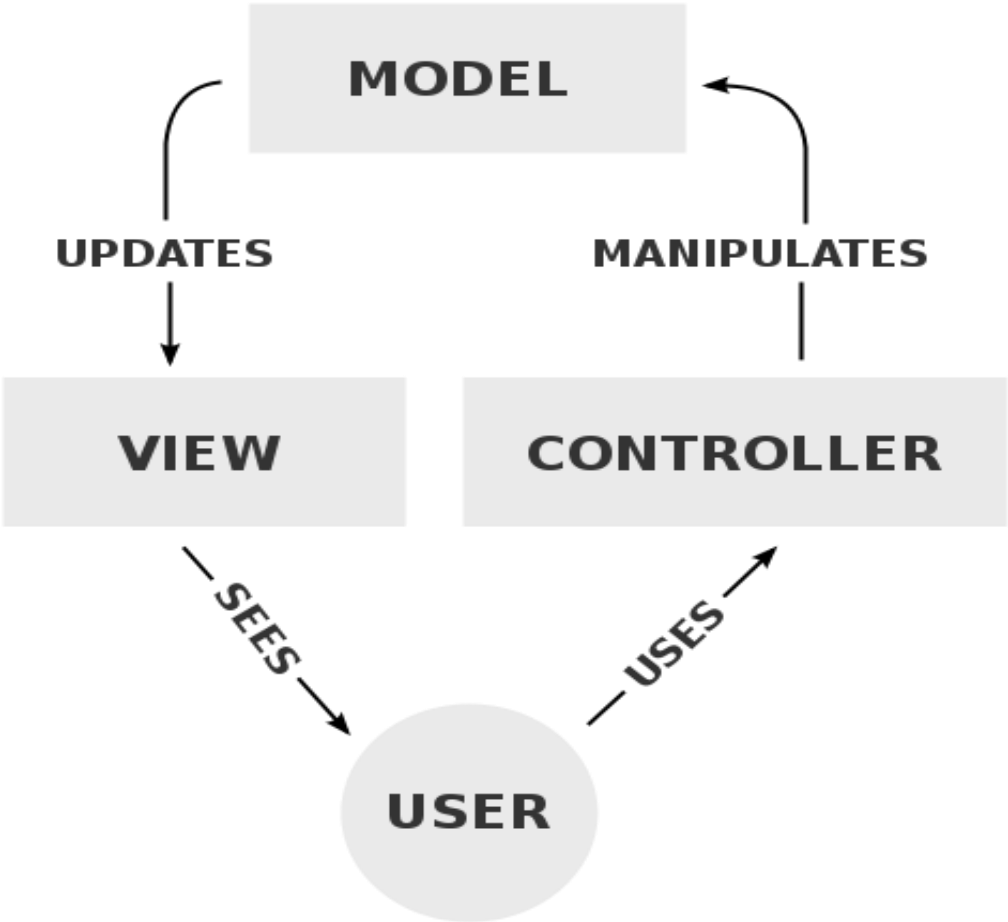
- The user interacts with the user interface in some way
  - e.g., user presses a button
- A controller handles the input event from the user interface, often via a registered handler or callback
- The controller accesses the model, possibly updating it in a way appropriate to the user's action
  - e.g., controller updates user's shopping cart

How MVC Works?

- A view uses the model to generate an appropriate user interface
  - e.g., view produces a screen listing the shopping cart contents

- The view gets its own data from the model. The model has no direct knowledge of the view.
  - However, the observer pattern can be used to allow the model to indirectly notify interested parties – potentially including views of a change

MVC with Controller



Describing Design Patterns

- Pattern Name and Classification: A descriptive and unique name that helps to identify and refer to the pattern
- Intent: A description of the goal behind the pattern and the reason for using it
- Also Know As: other well known name for the pattern
- Motivation: A scenario illustrates a design problem and how the pattern solve the problem
- Applicability: Situations in which this pattern can be applied, and example of poor designs that pattern can address

Describing Design Patterns

- Structure: A graphical representation of the classes in the pattern
  - Class diagrams and Interaction diagrams may be used for this purpose
- Participants: The classes and/or objects used in the pattern and their roles in the design
- Collaboration: A description of how classes and objects interact with each other
- Consequences: A description of the results, side effects, and tradeoffs caused by using the pattern

Describing Design Patterns

- Implementation: A description of an implementation of the pattern
- Sample Code: An illustration of how the pattern can be implemented using a programming language
- Known Uses: Examples of patterns found in real systems
- Related Patterns: Other design patterns are closely related to this one and discussion of the differences between the patterns

Design Patterns Classifications

- Purpose
  - Which reflects what a pattern does
  - Patterns can have either creational, structural, or behavioral purpose
- Creational patterns concern the process of object creation (5 design patterns)
- Structural patterns deal with the composition of classes or objects (7 design patterns)
- Behavioral patterns characterize the ways in which classes or objects interact and distribute responsibility (11 design patterns)

Design Patterns Classifications

- Scope
  - Specifies whether the pattern applies primarily to classes or to objects
- Class patterns (4 design patterns)
  - Deal with relationships between classes and their subclasses
  - These relationships are established through inheritance, so they are static—fixed at compile-time
- Object patterns (20 design patterns)
  - Deal with object relationships
  - It can be changed at run-time and are more dynamic

How Design Pattern Solve Design Problems

- Finding Appropriate Objects
- Determining Object Granularity
- Specifying Object Interfaces
- Specifying Object Implementations
- Putting Reuse Mechanisms to Work
- Relating Run-Time and Compile

1. Finding Appropriate Objects

- Object-Oriented design made up of objects
- Operations are the only way to change an object's internal data
  - Object internal state is encapsulation
- Object-Oriented design is decomposed into objects
  - Encapsulation, granularity, dependency, flexibility, performance, evolution, reusability, and so on

1. Finding Appropriate Objects

- Many objects in a design come from the design model
  - Many classes have no match in real world
- Strict modeling of the real world leads to a systems that reflect today's need but not flexible to to be used in the future
- Design patterns help to make a flexible design
  - Composite pattern helps to identify objects that does not have a physical match in real world at uniform level of abstraction
  - Algorithms are not part of nature but a crucial part of a flexible system. Strategy pattern describes how to implement interchangeable families of algorithms

2. Determining Object Granularity

- How to decide what should be an object?

- Design patterns describes how decompose an object into smaller objects
  - Abstract Factory and Builder patterns produce objects who's responsible for creating another objects

  - Visitor and Command patterns produce objects who's only responsible to implement request on another object or group of objects

3. Specifying Object Interfaces

- Object interface is the set of all operation signatures defined by object's operations
- The run-time association between an object and one of its operations is called a dynamic binding
- Design patterns help to identify interfaces and the relationship between them
  - Memento design patterns describes how to encapsulate and save the internal state of an object

4. Specifying Objects Implementations

- Programming to an interface, not an implementation
  - Inheritance is a mechanism to extend application functionality by reusing parent classes
  - Inheritance defined families of objects with identical interfaces: inherit from abstract class
  - Subclasses add or override operations: all subclasses are subtype of the interface

4. Specifying Objects Implementations

- Programming to an interface, not an implementation
  - + Clients remain unaware of the specific type of objects they use
  - + Clients remain unaware of the classes that implements the objects
  - + Reduce the implementation dependencies between subsystems

4. Specifying Objects Implementations

- Design patterns, such as creational patterns, ensure that your system is written in terms of interfaces, not implementations
  - Guide you to instantiate concrete classes
  - Give you different ways to associate an interfaces with its implementation

5. Putting Reuse Mechanism to Work

- Class inheritance vs. Object composition

- Class inheritance
  - + Class inheritance defined statically at compile-time
  - + It is easy to modify the implementation through overridden
  - - You cannot change the implementation inherited from the supper class at run-time
  - - Inheritance breaks encapsulation
    - Any change to parent's implementation will force the subclass to change (implementation dependencies)
    - Limits system flexibility and reusability

5. Putting Reuse Mechanism to Work

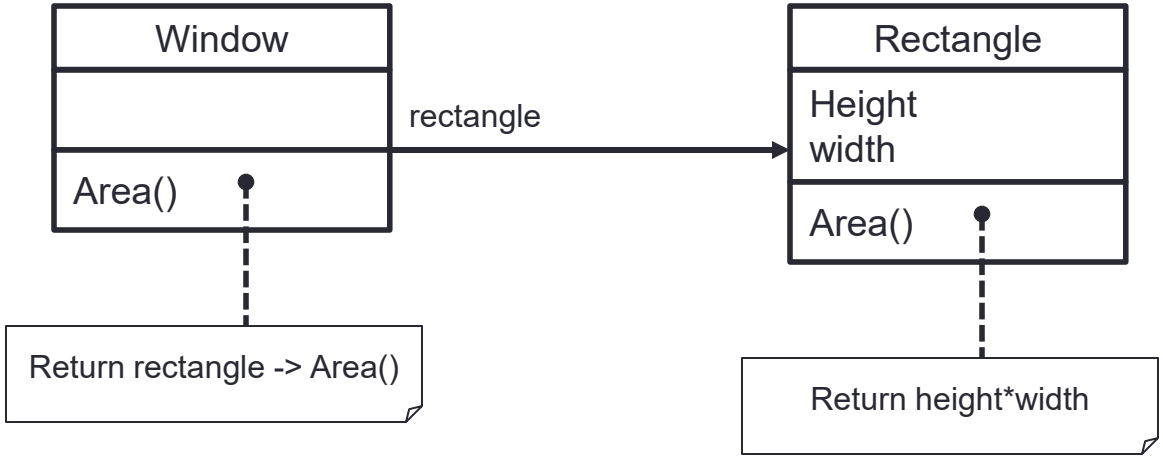
- Object Composition
  - Object composition is a way to combine simple objects to create complex one
  - Example: Coffee and CoffeeCup (Coffee can be referenced in CoffeeCup)
  - + Object composition defined dynamically at run-time
    - Through object referencing other objects
  - + Object composition are accessed through interfaces, thus don't break encapsulation
  - + Fewer implementation dependencies

5. Putting Reuse Mechanism to Work

- Favor object composition over class inheritance
- Design with object composition will have more objects and fewer classes
  - The system behavior depends on more interrelationships instead of being defined in one class

5. Putting Reuse Mechanism to Work

- Delegation
  - Two objects involved to handle a request



5. Putting Reuse Mechanism to Work

- Delegation: is a way to make composition powerful
  - + Delegation makes it easy to compose behaviors at run-time
  - - Makes software flexible through objects composition
  - - Dynamic software is harder to understand than static software
  - - Run-time inefficiencies

- Many design patterns used delegation
  - State, strategy, visitor, mediator

5. Putting Reuse Mechanism to Work

- Inheritance vs. parameterized types

- Parameterized types, also called genetics or templates
  - It lets you define a type without specifying all the other types. The unspecified types are supplied as parameters
  - E.g., List<Integers> = new ArrayList<Integer>();

  - + A way to compose behavior in OO system
  - + It lets you change the type the class can uses
  - - Cannot be changed at run-time

5. Relating Run-time and Compile-time Structures

- Software structure can be forced by the designer
- The relationship between objects and their types must be design with great care
  - They determine how good or bad is the run-time structure
- Some design patterns are useful for building complex run-time structures
  - Composite, decorator, observer

5. Designing for Change

- Consider how the system might need to change over lifetime
  - Might need to redesign the whole system in the future

- Design patterns help to ensure system can change over time by making the system robust of particular type of change
  - Tight coupling between classes: change in one class required changes to other classes (Abstract Factory, Bridge, Command)

  - Extending functionality by subclassing: define subclass requires understanding to the parent class. Use object composition is another way for combing behaviors but will make software hard to understand (Bridge, Chain of Responsibility, Composite)

Common Design Mistakes

- Not enough time has been spent to understand the underlying problem and its context, and as a consequence, you select a pattern that looks right, but is inappropriate for the solution required.
- Once the wrong pattern is selected, you refuse to see your error and force fit the pattern
  - It will result a poor or wrong fit.
- Sometimes a pattern is applied too exactly and the required adaptations for your problem space are not implemented.